



Mini-guida rapida a `release.py`

1 Dry-run (preview senza modifiche)

Mostra cosa succederebbe, senza scrivere nulla:

```
python release.py --patch --tag --commit --changelog
```

Cosa fa:

- Mostra la nuova versione calcolata da Git e incrementi automatici
- Preview colorata di `library.json` (vecchio → nuovo)
- Preview colorata di `CHANGELOG.md` in sezioni **Features/Fixes**
- Nessun file viene scritto
- Nessun commit o tag viene creato

⚠ Nota: tutte le modifiche vere richiedono `--go`.

2 Esecuzione reale (aggiorna file e Git)

```
python release.py --patch --tag --commit --changelog --go
```

Cosa fa:

1. Aggiorna `library.json` con la nuova versione
 2. Aggiorna `CHANGELOG.md` se ci sono commit rilevanti
 3. Esegue `git add` dei file aggiornati
 4. Commit automatico con messaggio "Release X.Y.Z"
 5. Tag annotato vX.Y.Z
 6. Push opzionale se aggiungi `--push`
-

3 Esempi di incremento versione

- Incremento patch (default per bugfix):

```
python release.py --patch --tag --commit --changelog --go
```

- Incremento minor (nuove funzionalità, compatibile con la vecchia API):

```
python release.py --minor --tag --commit --changelog --go
```

- Incremento major (cambiamenti incompatibili):

```
python release.py --major --tag --commit --changelog --go
```

- Versione manuale specifica:

```
python release.py 2.0.0 --tag --commit --changelog --go
```

4 Solo changelog

Se vuoi aggiornare solo il changelog senza toccare `library.json` o fare commit/tag:

```
python release.py --changelog --go
```

- Il changelog include solo commit “veri” (Features/Fixes)
 - Ignora `docs:`, `chore:`, `release`
 - Aggiorna solo `CHANGELOG.md`
-

5 Commit convenzionali consigliati

Per ottenere un changelog pulito:

- `feat:` → nuove funzionalità → finisce in **Features**
- `fix:` → correzioni bug → finisce in **Fixes**
- `chore:`, `docs:` → ignorati nel changelog
- Commit generici → finiscono in **Fixes**

Esempi:

```
git commit -m "feat: add colored output"
git commit -m "fix: crash on null buffer"
git commit -m "chore: update CI"
git commit -m "docs: update README"
git commit -m "refactor internal macros"
```

6 Dry-run vs Go

- **Dry-run** → mostra anteprima colorata, nessuna scrittura (`--go` mancante)
 - **Go** → esegue veramente tutte le modifiche e operazioni Git (`--go` presente)
-

Con questa mini-guida hai tutti gli scenari principali coperti.

Se vuoi, in futuro possiamo anche aggiungere:

- Snapshot giornalieri (`1.2.3+20260224`)

- Filtri avanzati per commit o template di changelog
 - Integrazione con CI/CD
-

Se vuoi, posso fare anche un **diagramma visivo del flusso di release** così vedi in un colpo d'occhio cosa succede in dry-run e in go. Vuoi che lo faccia?